

Yenepoya
Project – Low Level Design
on
Automated Pentesting for EdTech API
Ecosystems

Student Name: Mohammad Umar Faizee

Industry Mentor: Mr Shashank

Guided by: Ms Munisah

Table of Contents

1. Introduction.	6
1.1. Scope of the document.	6
1.2. Intended Audience	6
1.3. System overview.	6
2. System Design.	6
2.1. Application Design	6
2.2. Process Flow.	6
2.3. Information Flow.	7
2.4. Components Design	8
2.5. Key Design Considerations	8
2.6. API Catalogue.	8
3. Data Design.	8
3.1. Data Model	8
3.2. Data Access Mechanism	9
3.3. Data Retention Policies	9
3.4. Data Migration	9
4. Interfaces	9
5. State and Session Management	10
6. Caching	10
7. Non-Functional Requirements	10
7.1. Security Aspects	10
7.2. Performance Aspects	10
8. References	11

1. Introduction

1.1 Scope of the Document

This Low-Level Design (LLD) document provides granular, implementation-level specifications for the EdTech API Fuzzer. While the HLD addressed the "what" and "why" of the system, this document addresses the "how" — describing the exact functions, variables, data structures, and code-level logic used to build each module. Every specification in this document maps directly to a specific, existing line of code in the project files: `api_fuzzer.py`, `payloads.py`, and `demo_api.py`.

1.2 Intended Audience

This document is intended for software developers and technical reviewers who require a precise, code-aligned understanding of the implementation. It is directly useful for a viva/oral examination where a professor may ask: *"Show me exactly how your SQL Injection is detected"* or *"Explain exactly what happens when a thread encounters a timeout."*

1.3 System Overview

The system is composed of three Python files:

- **api_fuzzer.py** (614 lines): The main engine. It contains all logic for scanning, detection, reporting and terminal output.
- **payloads.py** (167 lines): A pure data file. Contains 9 dictionaries of attack vectors totalling over 80 unique payloads.
- **demo_api.py** (202 lines): A Flask-based intentionally vulnerable API running locally on <http://127.0.0.1:5050>.

2. System Design

2.1 Application Design

The application is initiated from `main()` at Line 575. The design follows a strict top-down control flow. `main()` calls `parse_args()` (Line 602) to read user arguments, then iterates through the `ENDPOINTS` list (Lines 96-128), calling `fuzz_endpoint()` for each one. The `fuzz_endpoint()` function in turn calls `get_baseline()` (Line 384) and then dispatches multiple threads via `ThreadPoolExecutor` (Line 466), each of which calls `process_request()` (Line 423).

Class Definitions:

- Severity (Line 42): A static class acting as an Enum with four levels: HIGH, MEDIUM, LOW, INFO.
- Color (Line 69): A static class containing ANSI escape codes (\033[91m for Red, etc.) for coloring terminal output.

Global Configuration (Lines 84-93):

```
python
DEFAULT_CONFIG = {
    "base_url": "http://127.0.0.1:5050",
    "timeout": 8,
    "delay": 0.3,
    "output_csv": "results.csv",
}
TIME_FAST = 0.5 # seconds — green
TIME_SLOW = 2.0 # seconds — red, flagged
```

2.2 Process Flow

Step 1 — parse_args() (Line 602): argparse.ArgumentParser() registers the following arguments:

- --url → default "<http://127.0.0.1:5050>"
- --category → default "all" (maps to ALL_PAYLOADS dict)
- --timeout → default 8 seconds per request
- --threads → default 5 concurrent threads
- --output → default "results.csv"

Step 2 — get_baseline() (Line 384): The function constructs a valid "clean" request:

```
python
body = {
    "name": "Normal Student", "email": "norm@test.edu",
    "grade": 80, "course": "CS101", "username": "guest", "password": "password"
}
```

It records three metrics: status_code, response_time_s (via time.time()), and response_size (via len(resp.text)).

Step 3 — fuzz_endpoint() (Line 457): Calls ThreadPoolExecutor(max_workers=threads). It submits one future per payload using executor.submit(process_request, ...). Results are collected lazily using as_completed(futures).

Step 4 — process_request() (Line 423): For POST requests, it builds a standard JSON body and injects the malicious payload into the fuzz_field key (e.g., "name"):

python

```
body[fuzz_field] = payload # e.g., body["name"] = "' OR '1'='1'"
```

For GET requests, it builds query parameters replacing "__FUZZ__" with the payload:

python

```
params = {k: (payload if v == "__FUZZ__" else v)
          for k, v in endpoint.get("params", {}).items()}
```

Step 5 — is_interesting() (Line 190): The detection engine. Detailed below.

Step 6 — write_csv() (Line 486) + generate_html_report() (Line 283): Writes final output files.

2.3 Information Flow

Data flows through the following chain without any shared mutable state between threads (thread-safe by design):

```
CLI args → ENDPOINTS config → get_baseline() → ThreadPoolExecutor
→ process_request() [per thread] → is_interesting() → result dict
→ results list → write_csv() + generate_html_report()
```

The only shared resource is the results list, which is extended using `all_results.extend(res)` at Line 587 — this is safe because `extend` is only called after all threads in a batch complete (after the `with ThreadPoolExecutor` block exits).

2.4 Components Design

is_interesting() — The Detection Engine (Lines 190-272):

This is the most critical function. It operates in three stages using a numeric score variable:

Stage 1 — Critical Indicators (Direct Proof):

python

```
if code == 500:
```

```
    score += 85      # Server crash is near-certain vulnerability
```

```
for pattern in SQL_ERRORS:
```

```
    if pattern in resp_text:
```

```
        score += 100 # Database error is definitive proof
```

The `SQL_ERRORS` list (Line 58) contains 11 specific patterns including `"sqlite3.error"`, `"sql syntax"`, `"ora-"`, `"sqlstate"`.

Stage 2 — Anomaly Indicators (Statistical Baseline Comparison):

```
python
if str(code) != str(b_code):
    score += 60      # Status code changed from baseline
diff = abs(size - b_size) / b_size
if diff > 1.0:
    score += 50      # >100% size change
elif diff > 0.3:
    score += 30      # >30% size change
if elapsed >= TIME_SLOW and elapsed > (b_time * 4):
    score += 70      # Timing anomaly (4x slower than baseline)
```

Stage 3 — Noise Filtering (Smart Reflection):

```
python
if any(tag in payload for tag in "<>\"'()[]{}%$^&*"):
    score += 15      # Only flag reflections with dangerous chars
```

Severity Mapping (Lines 262-272):

```
python
confidence = min(100, score)
if confidence >= 90: → Severity.HIGH
if confidence >= 40: → Severity.MEDIUM
if confidence >= 15: → Severity.LOW
else: → Severity.INFO
```

mutate_payload() (Line 274): Returns 5 variations of every payload to bypass simple sanitization:

```
python
[payload, payload.upper(), payload + "", "" + payload, " " + payload]
```

2.5 Key Design Considerations:

Thread Safety: The ThreadPoolExecutor is used inside fuzz_endpoint() with a with block. The futures dictionary maps each Future object to its payload. as_completed() yields futures as they resolve, not in submission order — this is why the terminal output may show results out of order.

Timeout Logic: Every requests.request() call (Line 440) is wrapped in a try-except Exception block. A timeout causes requests.exceptions.Timeout to be raised, which is caught, and the result is stored with "status_code": "ERROR". This prevents any single hanging request from blocking the thread pool.

Payload Categorization: The CATEGORIES dictionary (Line 130) maps CLI string arguments to the imported payload dictionaries:

```
python
CATEGORIES = {
    "sql_injection": SQL_INJECTION, # 10 payloads
    "xss":           XSS,           # 10 payloads
    "all":           ALL_PAYLOADS,  # 80+ payloads
    ...
}
```

2.6 API Catalogue (Exact Route Specifications)

From ENDPOINTS list (Lines 96-128) and demo_api.py:

Name	URL	Method	Fuzz Field	Vulnerability
Generic Web Test	base}/post	POST	fuzz_data	Generic echo test
Add Student	base}/api/students	POST	name	No input validation (Line 103 demo_api)
Search Students	base}/api/students/search	GET	'name=	Raw string concat SQL query (Line 30 demo_api)
Login	base}/api/login	POST	username	Auth bypass — any truthy username (Line 171 demo_api)
Admin Users	base}/api/admin/users	GET	—	No auth check, exposes secrets (Line 82 demo_api)

3. Data Design

3.1 Data Model

Payload Structure (payloads.py): 9 dictionaries, each with string keys and attack-vector values:

python

```
SQL_INJECTION = {
    "sqli_classic": "' OR '1'='1",      # Boolean logic bypass
    "sqli_drop":    "'; DROP TABLE students; --", # Destructive DDL
    "sqli_union":   "' UNION SELECT null, username, password FROM users --",
    "sqli_sleep":   "'; WAITFOR DELAY '0:0:5'; --", # Time-based
    "sqli_blind":   "' AND 1=1 --",      # Boolean-based blind
    "sqli_stacked": "1; SELECT * FROM information_schema.tables --",
    ...
}
```

Result Dictionary Schema (built in process_request(), Line 442):

python

```
result = {
    "endpoint":      "Add Student (POST)",
    "category":      "sql_injection",
    "label":         "sqli_drop",
    "payload":       "'; DROP TABLE students; --",
    "status_code":   500,
    "response_time_s": 0.003,
    "response_size": 214,
    "response_full": '{"status": "error", "detail": "..."}',
    "response_snippet": '{"status": "error"',
    "severity":      "HIGH",
    "reason":        "Server Crash (500) (Conf: 85%)",
    "evidence":      "Status 500 returned"
}
```

3.2 Data Access Mechanism

All target API data is accessed via `requests.request()` (Line 440):

python

```
resp = requests.request(  
    method, # "GET" or "POST"  
    url,    # e.g., "http://127.0.0.1:5050/api/students/search"  
    json=body,    # serialized as application/json  
    params=params, # URL-encoded for GET  
    headers={"Content-Type": "application/json"},  
    timeout=timeout # default 8 seconds  
)
```

Response data is accessed from `resp.status_code`, `resp.text`, and `len(resp.text)`.

3.3 Data Retention Policies

Volatile (RAM): All results exist in the `all_results` list during execution. When the script terminates, this is cleared by the OS.

Persistent (Disk):

- `results.csv` — written by `write_csv()` using `csv.DictWriter` with 9 fieldnames (Line 487).
- `report.html` — written by `generate_html_report()` using Python's `open()` with UTF-8 encoding (Line 380).

3.4 Data Migration

The CSV schema uses universal column names (`endpoint`, `status_code`, `payload`, `reason`, etc.), ensuring portability to any data analysis tool — Excel, Python Pandas, or Elasticsearch — without transformation.

4. Interfaces

4.1 CLI Interface (parse_args(), Line 602)

Full argument specification:

```
python3 api_fuzzer.py
[--url http://127.0.0.1:5050]      Target base URL
[--category sql_injection|xss|all] Payload category
[--timeout 8]                  Request timeout in seconds
[--threads 5]                  Concurrent thread count
[--output results.csv]         Output file path
[--list-categories]            Print all categories and exit
```

4.2 HTML Report Interface (generate_html_report(), Line 283)

The function takes four parameters: results (list), summary (dict), path (string), target_url (string). It generates a self-contained HTML file with:

- A metadata header (Author, Target URL, Timestamp)
- A 4-column CSS grid showing: Total Requests, High Risk, Medium Risk, Low Risk counts
- A <table> with severity badges, endpoint names, vulnerability reasons, and payload evidence
- An academic disclaimer footer

4.3 CSV Export Interface (write_csv(), Line 486)

```
python
fieldnames = ["endpoint", "category", "label", "payload",
              "status_code", "response_time_s", "response_size",
              "reason", "response_snippet"]
```

Uses csv.DictWriter with extrasaction='ignore' to safely skip internal keys like response_full and severity that are not needed in the CSV.

5. State and Session Management

The scanner is **stateless by design**. The `process_request()` function (Line 423) creates a fresh headers dictionary and body dictionary on every invocation — no cookies, no session objects, no shared state between threads. Each thread has its own local namespace.

For the `demo_api.py` target, Flask's development server handles each request independently on its own thread. The SQLite database (":memory:") is a single shared connection (`_conn`) established at module level (Line 37), which is safe for read-heavy workloads but would cause `sqlite3.OperationalError` under concurrent writes — this is intentionally one of the vulnerabilities demonstrated by `sqli_stacked`.

6. Caching

Baseline Cache (per-endpoint): In `fuzz_endpoint()` (Line 461):

```
python
```

```
baseline = get_baseline(endpoint, base_url, timeout)
```

This dictionary is captured once and passed as a parameter to every `process_request()` call via the `executor.submit()` call (Line 467). It is read-only inside the threads — never mutated — making this cache inherently thread-safe without needing a lock.

The cache stores three values: `status_code`, `response_time_s`, and `response_size`. The `is_interesting()` function reads from this cache at Lines 228-250 to calculate proportional deviations.

7. Non-Functional Requirements

7.1 Security Aspects

The `SQL_ERRORS` list (Line 58) contains 11 database driver-specific error strings. This means the scanner can detect errors from SQLite ("sqlite3.error"), MySQL ("mysql_fetch", "sql syntax"), Oracle ("ora-"), PostgreSQL ("postgres"), and MSSQL ("sqlstate").

The `demo_api.py` intentionally exposes 8 coded vulnerabilities:

1. No length validation on name field (Line 103)
2. Unhandled `KeyError` on missing fields (Line 103-104)
3. No grade range validation (Line 108)
4. Raw string concatenation in SQL query (Line 130)
5. Full error message exposed in 500 response (Line 143)
6. Hardcoded credentials with no rate-limiting (Line 169)
7. Auth bypass — any non-empty username gets a token (Line 171)
8. No authentication on admin route, exposes `admin_secrets` table (Line 182-193)

7.2 Performance Aspects

Default configuration: --threads 5. The ThreadPoolExecutor with 5 workers processes all 5 endpoints $\times$ 10 payloads (50 total) in approximately 1-2 seconds on a local machine. Without threading, this would take approximately $50 \times 0.3\text{s}$ (delay) = 15 seconds.

The `format_time()` function (Line 409) provides color-coded timing feedback: green for $< 0.5\text{s}$, yellow for $0.5\text{-}2.0\text{s}$, red for $> 2.0\text{s}$ — giving the user instant visual performance feedback in the terminal.

8. References

1. **Python concurrent.futures Documentation** — `ThreadPoolExecutor`, `as_completed()` — used at Lines 31, 466-469 of `api_fuzzer.py`.
2. **Python requests Library Documentation** — `requests.request()`, `requests.exceptions.Timeout` — used at Line 440.
3. **Flask Documentation** — Route decorators `@app.route()`, `request.get_json()`, `request.args.get()` — used throughout `demo_api.py`.
4. **Python sqlite3 Module Documentation** — `connect()`, `cursor.execute()`, `executescript()` — used at Lines 33-59 of `demo_api.py`.
5. **OWASP SQL Injection Prevention Cheat Sheet** — Basis for `SQL_ERRORS` signature list (Line 58) and `SQL_INJECTION` payload category in `payloads.py`.
6. **OWASP XSS Prevention Cheat Sheet** — Basis for the XSS reflection detection logic at Line 220-224 of `api_fuzzer.py`.